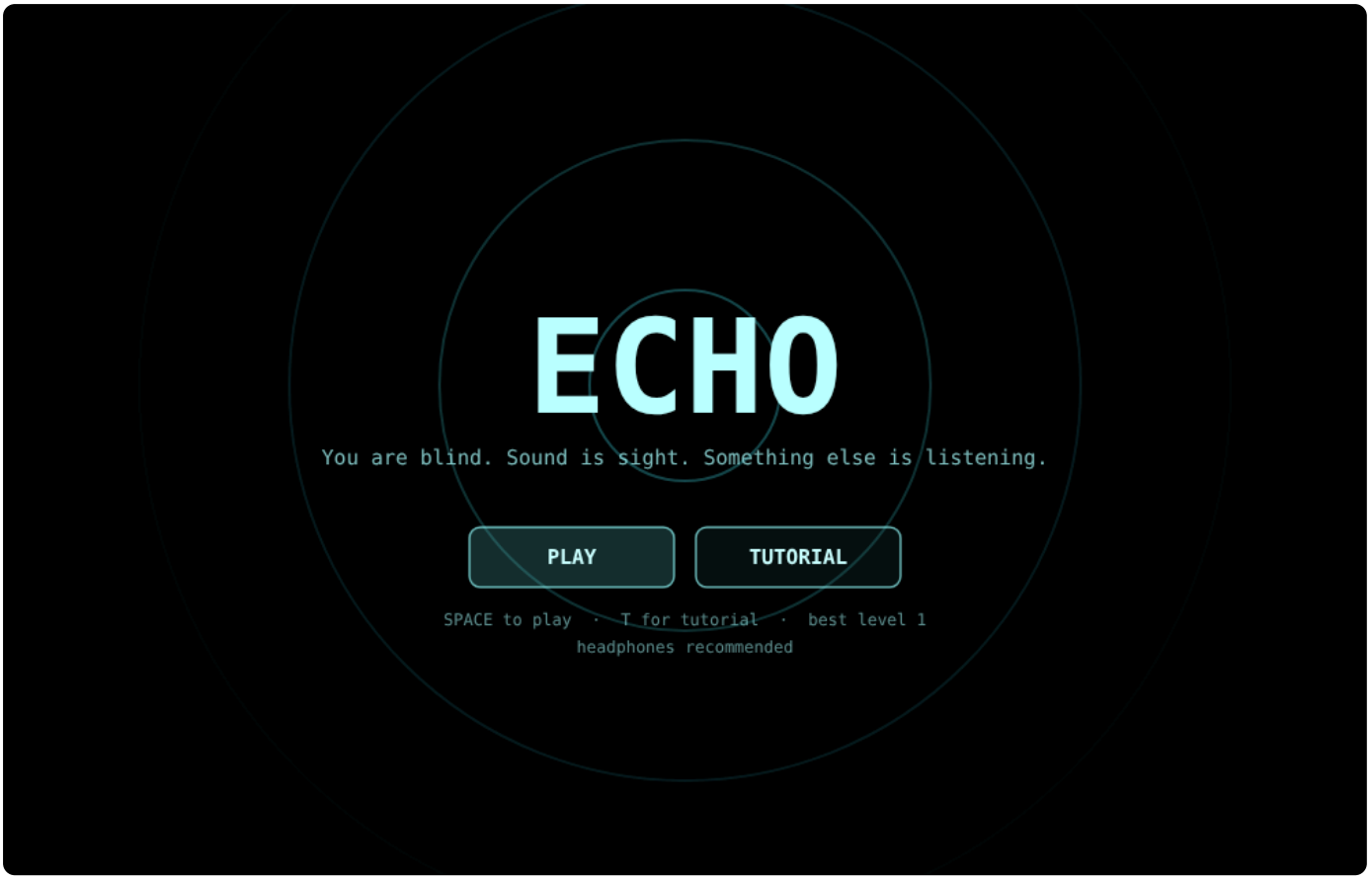
ECHO

The complete code, explained line by line



A small stealth game written in one HTML file, with no engine, no libraries and no image or sound files. This guide walks through all 708 lines of `index.html`, in order, in plain language.

By Praise Adejubu · 2026 · MIT licence

How to use this guide

The code is shown in small groups of one to a few lines. Each group has its **real line numbers** from `index.html`, so you can open the file next to this guide and follow along. Under each group is a plain-language explanation of what those lines do and why they are there.

You do not need to read it all in one go. The first few pages explain the big ideas. After that, each numbered section matches a part of the program, so you can jump to whichever bit you are curious about.

> **Tip:** the fastest way to learn from this is to change one number (for example the hearing range on the `HEAR` line), save, refresh the browser, and see what happens.

The game in a nutshell

The screen is completely black. You cannot see. The only way to see is to make a sound: the sound travels outwards, bounces off walls, and the echoes briefly draw the shape of the room. The catch is that the maze is full of creatures who are also blind, and they hunt by sound.

- **Sonar pulse** (Space or tap): lets you see far (700 pixels), but creatures hear it from 420 pixels away.
- **Quiet click** (E or two-finger tap): only shows walls close to you (170 pixels), and creatures only hear it from 110 pixels.
- **Walking** (WASD, arrows or drag) is completely silent.
- **Running** (Shift, or drag far) is fast, but every footstep is a noise heard from 200 pixels.
- Reach the **gold exit** to go to the next level. Each level is bigger and has more creatures.
- You have **3 lives**. If a creature touches you, you lose one and restart the same maze. Lose all 3 and it is game over, back to level 1.

The rules that make the creatures fair

The creatures follow simple, consistent rules, and the game shows you every one of them on screen:

1. **They only hear, never see.** If you are silent, they do not know where you are.
2. **They go to where the sound was, not to where you are.** When you make a noise, a red X appears on that spot. Every creature that heard it walks to the X. If you have moved away, they find nothing.
3. **You can see how far your noise travels.** Each noise makes a dashed red ring. Any creature inside the ring hears it, at the moment the ring reaches it.
4. **You can see them react.** A creature that hears you glows red with a "!" for a moment and growls.
5. **You can see them coming.** Hunting creatures stomp, leaving red ripples and thuds. Wandering ones shuffle, which you notice when they are close. Your heartbeat speeds up and

the screen edges turn red as one gets near.

6. **You are told why you were caught.** "It heard your sonar", "It heard you running", "It heard your click", or "It bumped into you" if you were simply unlucky.





How the program is put together

Almost every game, big or small, is built around the same idea: a **game loop**. About 60 times a second the program does two things:

1. **Update:** move the clock forward a tiny amount, read the controls, move the player and creatures, and check what has happened (caught, escaped, tutorial step done).
2. **Draw:** clear the screen and paint everything again in its new position.

Everything else is a helper for one of those two jobs. The file is laid out in this order:

Section	What it does
1. The HTML page	The bare web page: a black background and one canvas to draw on.
2. Setting up the canvas	Makes the canvas fill the window and stay sharp on high-resolution screens.
3. Game settings	All the numbers that control the feel of the game, in one place.
4. Seeded random numbers	Random numbers you can replay, so a maze can be rebuilt exactly.
5. The maze	Builds random mazes, and loads the hand-drawn tutorial map.
6. Path finding	Works out the shortest route between two tiles. Creatures use this.
7. Sound effects	Every sound is created live from simple waves.
8. Game state	The variables that remember everything about the current moment.
9. Starting a game and a level	Sets up a fresh run, or a fresh level.
10. The tutorial	The tutorial map and its seven guided steps.
11. Echolocation	Fires rays and turns them into echoes you can see.
12. Noise	Decides which creatures hear you, and sends them to the sound.

13. Controls	Keyboard, mouse and touch.
14. Updating the world	The "update" half of the loop: movement, creature brains, lives, exits.
15. Drawing the world	The "draw" half of the loop.
16. Screen text and buttons	Hearts, messages, the tutorial box, the title and game over screens.
17. The game loop	Ties it all together and keeps it running.

Words worth knowing

Word	Meaning here
Canvas	A rectangle on a web page that JavaScript can draw pixels onto.
Tile	One square of the maze grid, 40 by 40 pixels. Each tile is either wall or floor.
Frame	One picture. The game draws about 60 frames a second.
dt (delta time)	How many seconds passed since the last frame, usually about 0.016. Speeds are multiplied by dt so the game runs at the same pace on fast and slow computers.
Ray / ray casting	Following an imaginary straight line from a point until it hits something.
DDA	"Digital Differential Analyser": a quick way to walk a ray through a grid one tile at a time.
BFS	"Breadth-first search": flooding outward from a tile to find the shortest route through the maze.
Seed	A starting number for a random number generator. Same seed, same "random" results.
Web Audio	The browser's built-in sound system. It can generate tones from nothing.
Oscillator	A sound generator that produces a steady wave at a chosen pitch.
Gradient	A smooth blend between colours, used here for glows.
Alpha	Transparency, from 0 (invisible) to 1 (solid).
Arrow function	A short way to write a function: `x => x * 2` means "take x, give back x times 2".
Template string	Text in backticks where `${...}` inserts a value, for example `` `LEVEL ${level}` ``.
const / let	`const` names a value that will not be replaced; `let` names one that can change.

1. The HTML page

```
1 <!doctype html>
```

Line 1. Tells the browser this is a modern HTML5 page, so it follows the current standard rules.

```
2 <html lang="en">
```

Line 2. Opens the page and says its text is in English. This helps screen readers and search engines.

```
3 <head>
```

Line 3. The head holds settings about the page. Nothing in it is drawn on screen.

```
4 <meta charset="utf-8">
```

Line 4. Use UTF-8 text encoding so every character (including symbols like the middle dot) displays correctly.

```
5 <meta name="viewport" content="width=device-width,initial-scale=1">
```

Line 5. Makes the page fit phone screens properly, instead of showing a tiny zoomed-out desktop view.

```
6 <title>ECHO</title>
```

Line 6. The name shown on the browser tab.

```
7 <style>
8   html,body{margin:0;height:100%;background:#000;overflow:hidden}
9   canvas{display:block;touch-action:none}
10 </style>
```

Lines 7 to 10. A small block of CSS styling. Line by line: remove the default page margin, make the page fill the window, paint the background black and hide scrollbars; then make the canvas a plain block (no stray gap underneath) and tell touch screens not to scroll or zoom when you drag on it.

```
11 </head>
12 <body>
```

Lines 11 to 12. Close the head and open the body, which is the visible part of the page.

```
13 <canvas id="c"></canvas>
```

Line 13. The canvas is the one and only drawing surface. Every pixel of the game is painted onto it by the JavaScript below. The id "c" lets the script find it.

```
14 <script>
```

Line 14. Everything from here to the closing script tag is the game itself, written in plain JavaScript.

```
15 'use strict';
```

Line 15. Strict mode makes JavaScript stop with an error on common mistakes (such as using a variable that was never declared) instead of quietly carrying on and causing odd bugs later.

2. Setting up the canvas

```
17 const cv = document.getElementById('c'), cx = cv.getContext('2d');
```

Line 17. Find the canvas on the page (cv) and get its 2D drawing tool (cx). Every drawing command in the game goes through cx.

```
18 let DPR = 1, VW = 0, VH = 0;
```

Line 18. DPR is the screen's pixel density (1 on older monitors, 2 or 3 on phones and Retina screens). VW and VH are the window's width and height in normal CSS pixels.

```
19 function resize() {
```

Line 19. resize() makes the canvas match the window.

```
20   DPR = Math.min(2, window.devicePixelRatio || 1);
```

Line 20. Read the pixel density, capped at 2 so very dense phone screens do not slow the game down.

```
21   VW = innerWidth; VH = innerHeight;
```

Line 21. Store the window size.

```
22   cv.width = VW * DPR; cv.height = VH * DPR;
```

Line 22. Give the canvas enough real pixels for a sharp image: window size times density.

```
23   cv.style.width = VW + 'px'; cv.style.height = VH + 'px';
24 }
```

Lines 23 to 24. But display it at the normal window size. The extra pixels make it crisp rather than bigger.

```
25 addEventListener('resize', resize); resize();
```

Line 25. Run resize() whenever the window changes size (or a phone rotates), and once right now at start-up.

3. Game settings (constants)

```
27  const TILE = 40;
```

Line 27. Each maze square, called a tile, is 40 by 40 pixels.

```
28  const SOUND = 520;
```

Line 28. The speed of sound in this game, in pixels per second. Echoes and noise rings travel at this speed, so far-away things show up a moment later than near things, just like real echoes.

```
29  const SIGHT = { pulse: 700, click: 170, step: 90 };
```

Line 29. How far each kind of sound lets YOU see, in pixels: the big sonar pulse, the quiet click, and a running footstep.

```
30  const HEAR = { pulse: 420, click: 110, step: 200 };
```

Line 30. How far each kind of sound can be HEARD by creatures, in pixels. This is the key rule of the game. A pulse lets you see 700 px but creatures only hear it within 420 px, so it is useful but risky. Walking is not listed at all because walking is completely silent.

```
31  const RAYS = { pulse: 320, click: 140, step: 50 };
```

Line 31. How many rays each kind of sound sends out. More rays give a more detailed picture of the walls.

```
32  const WALK = 115, RUN = 190, MAX_LIVES = 3;
```

Line 32. Your walking and running speed in pixels per second, and how many lives you get per game.

```
33  const DIRS = [[1, 0], [-1, 0], [0, 1], [0, -1]];
```

Line 33. The four directions you can step in a grid (right, left, down, up). The path finder uses these.

```
34  const WALL = 1, EXIT = 2, FOE = 3, DIMWALL = 4;
```

Line 34. Readable names for what an echo can hit: a wall, the exit, a creature (foe), or a wall found by a quiet sound, which is drawn dimmer.

```
35  const COL = { 1: 'hsl(185,100%,65%)', 2: 'hsl(45,100%,60%)', 3: 'hsl(0,100%,62%)', 4: 'hsl(185,60%,50%)'
    };
```

Line 35. The colour for each of those: cyan walls, gold exit, red creatures, darker cyan for quiet echoes. The numbers match the names above.

```
36  const RED = COL[FOE];
```

Line 36. A short name for the red colour, which is also used for noise rings and hearts.

```
37  const FONT = 'ui-monospace, Menlo, Consolas, monospace';
```

Line 37. The font for all text: whichever monospace (fixed-width) font the device has.

4. Seeded random numbers

```
39 function makeRng(seed) {
```

Line 39. makeRng() builds a random number generator from a starting number called a seed. The method is known as mulberry32. The same seed always produces the same sequence, which is how the game gives you the exact same maze again after you lose a life.

```
40   return function () {
```

Line 40. It returns a small function. Each call to it produces the next random number.

```
41     seed |= 0; seed = seed + 0x6D2B79F5 | 0;
```

Line 41. Move the seed forward by a large fixed amount. The "| 0" keeps it a whole 32-bit number.

```
42     let r = Math.imul(seed ^ seed >>> 15, 1 | seed);
43     r = r + Math.imul(r ^ r >>> 7, 61 | r) ^ r;
```

Lines 42 to 43. Scramble the bits thoroughly using shifts (>>>), exclusive-or (^) and 32-bit multiplication (Math.imul).

```
44     return ((r ^ r >>> 14) >>> 0) / 4294967296;
45   };
46 }
```

Lines 44 to 46. Turn the result into a decimal between 0 and 1, the same range as Math.random().

5. The maze

```
48 let G, GW, GH, R, openTiles = [];
```

Line 48. G is the grid, one number per tile (1 = wall, 0 = floor). GW and GH are its width and height in tiles. R is the random generator for the current level. openTiles lists every floor tile.

```
49 function genMaze(w, h, loops) {
```

Line 49. genMaze() builds a random maze w tiles wide and h tiles tall. "loops" is how many extra walls to try knocking down afterwards.

```
50   G = new Uint8Array(w * h).fill(1); GW = w; GH = h;
```

Line 50. Start with every tile as wall, and remember the size.

```
51   const stack = [[1, 1]]; G[w + 1] = 0;
```

Line 51. This is the "recursive backtracker" method. Keep a stack (a pile) of cells, starting with tile (1, 1), which is opened up. Grid tile (x, y) is stored at position y * w + x.

```
52   while (stack.length) {
```

Line 52. Keep going until the stack is empty, which means every cell has been carved.

```
53     const [x, y] = stack[stack.length - 1];
```

Line 53. Look at the cell on top of the pile.

```
54     const options = [[2, 0], [-2, 0], [0, 2], [0, -2]].filter(([dx, dy]) => {
55       const nx = x + dx, ny = y + dy;
56       return nx > 0 && ny > 0 && nx < w - 1 && ny < h - 1 && G[ny * w + nx] === 1;
57     });
```

Lines 54 to 57. List its neighbours two tiles away (right, left, down, up) that are inside the outer border and still solid, meaning not visited yet.

```
58     if (!options.length) { stack.pop(); continue; }
```

Line 58. No unvisited neighbours means a dead end, so back up by taking this cell off the pile.

```
59     const [dx, dy] = options[Math.floor(R() * options.length)];
```

Line 59. Otherwise pick one neighbour at random.

```
60     G[(y + dy / 2) * w + x + dx / 2] = 0;
61     G[(y + dy) * w + x + dx] = 0;
```

Lines 60 to 61. Open the wall tile in between, and the neighbour itself. This carves a corridor.

```
62     stack.push([x + dx, y + dy]);
63   }
```

Lines 62 to 63. Carry on from the new cell.

```
64    for (let i = 0; i < loops; i++) {
```

Line 64. A pure backtracker maze has only one route between any two points, so a creature in your corridor would trap you. Knocking out extra walls creates loops, which give you escape routes.

```
65      const x = 1 + Math.floor(R() * (w - 2)), y = 1 + Math.floor(R() * (h - 2)), c = y * w + x;
```

Line 65. Pick a random tile inside the border. c is its position in the grid.

```
66      if (G[c] !== 1) continue;
```

Line 66. Skip it if it is already floor.

```
67      const across = G[c - 1] === 0 && G[c + 1] === 0 && G[c - w] && G[c + w];
68      const down = G[c - w] === 0 && G[c + w] === 0 && G[c - 1] && G[c + 1];
69      if (across || down) G[c] = 0;
70    }
```

Lines 67 to 70. Only remove a wall that sits neatly between two corridors: floor on its left and right with wall above and below (across), or the other way round (down). This avoids making messy open spaces.

```
71    listOpenTiles();
72  }
```

Lines 71 to 72. Finally, record every floor tile.

```
73  function listOpenTiles() {
74    openTiles = [];
75    for (let i = 0; i < G.length; i++) if (!G[i]) openTiles.push(i);
76  }
```

Lines 73 to 76. listOpenTiles() fills openTiles with the position of every floor tile. Creatures pick wandering spots from this list.

```
77  function loadMap(rows) {
```

Line 77. loadMap() turns a hand-drawn map (a list of text rows) into the grid. The tutorial uses it. "#" is wall and anything else is floor. Letters such as P, C and X mark special spots, and their positions are collected in "marks" and returned.

```
78    GH = rows.length; GW = rows[0].length;
79    G = new Uint8Array(GW * GH);
80    const marks = {};
```

Lines 78 to 80. The map's size comes from the number of rows and the length of the first row.

```
81    for (let y = 0; y < GH; y++) {
82      for (let x = 0; x < GW; x++) {
83        const ch = rows[y][x];
```

Lines 81 to 83. Visit every character in every row.

```
84        G[y * GW + x] = ch === '#' ? 1 : 0;
```

Line 84. A "#" becomes wall, everything else floor.

```
85        if (ch !== '#' && ch !== '.') marks[ch] = { tx: x, ty: y };
86      }
87    }
88    listOpenTiles();
89    return marks;
90  }
```

Lines 85 to 90. Remember where each letter is.

```
91  function solid(tx, ty) { return tx < 0 || ty < 0 || tx >= GW || ty >= GH || G[ty * GW + tx] === 1; }
```

Line 91. solid() answers "is tile (tx, ty) a wall?". Anything outside the map also counts as wall, so nothing can ever leave the maze.

```
92  const tileOf = v => Math.floor(v / TILE);
93  const mid = tile => (tile + 0.5) * TILE;
```

Lines 92 to 93. Two helpers. tileOf() turns a pixel position into a tile number. mid() gives the pixel position of a tile's centre.

6. Path finding

```
 95 function bfs(sx, sy) {
```

Line 95. bfs() is a "breadth-first search". From one starting tile it spreads out step by step like a flood, recording how many steps away each tile is (dist) and which tile it was reached from (prev).

```
 96   const dist = new Int32Array(GW * GH).fill(-1), prev = new Int32Array(GW * GH).fill(-1);
```

Line 96. Two arrays with one slot per tile, both filled with -1, meaning "not reached yet".

```
 97   const queue = [sy * GW + sx]; dist[queue[0]] = 0;
```

Line 97. The queue of tiles to process starts with just the starting tile, which is 0 steps away.

```
 98   for (let head = 0; head < queue.length; head++) {
```

Line 98. Work through the queue in order. New tiles join at the back, so nearer tiles are always handled before further ones.

```
 99     const c = queue[head], x = c % GW, y = (c / GW) | 0;
```

Line 99. Turn the tile's position back into x and y.

```
100     for (const [dx, dy] of DIRS) {
101       const nx = x + dx, ny = y + dy, n = ny * GW + nx;
```

Lines 100 to 101. Check each of the four neighbours.

```
102       if (nx < 0 || ny < 0 || nx >= GW || ny >= GH || G[n] || dist[n] >= 0) continue;
```

Line 102. Ignore it if it is off the map, a wall, or already reached.

```
103       dist[n] = dist[c] + 1; prev[n] = c; queue.push(n);
104     }
105   }
106   return { dist, prev };
107 }
```

Lines 103 to 107. Otherwise it is one step further than the current tile. Note where it came from and add it to the queue.

```
108 function findPath(sx, sy, tx, ty) {
109   const { prev, dist } = bfs(tx, ty);
110   let c = sy * GW + sx;
```

Lines 108 to 110. findPath() returns the list of tiles to walk through to get from (sx, sy) to (tx, ty). It floods out from the target, so following the "prev" links from the start leads straight to the target by the shortest route.

```
111   if (dist[c] < 0) return [];
112   const path = [], goal = ty * GW + tx;
```

Lines 111 to 112. If the flood never reached the start, there is no route, so return an empty path.

```
113   while (c !== goal) { c = prev[c]; path.push(c); }
114   return path;
115 }
```

Lines 113 to 115. Hop from tile to tile towards the goal, collecting each one.

7. Sound effects (made from scratch)

```
117 let AC = null;
```

Line 117. AC is the Web Audio context, the browser's built-in sound engine. Browsers only allow sound after the player presses something, so it starts empty.

```
118 function initAudio() {
119   if (!AC) { try { AC = new (window.AudioContext || window.webkitAudioContext)(); } catch (e) { } }
120   if (AC && AC.state === 'suspended') AC.resume();
121 }
```

Lines 118 to 121. initAudio() creates the sound engine the first time it is called, and wakes it if the browser has paused it. The try/catch means a browser without Web Audio still runs the game, just silently.

```
122 function tone(f0, f1, dur, vol, type = 'sine', delay = 0) {
```

Line 122. tone() plays one synthesised sound. f0 and f1 are the starting and ending pitch in hertz, dur is the length in seconds, vol is the loudness, type is the wave shape (sine is smooth, square and sawtooth are buzzy, triangle is in between) and delay is how long to wait before playing.

```
123   if (!AC || vol <= 0.0005) return;
```

Line 123. Do nothing if there is no sound engine, or if the sound would be too quiet to hear anyway.

```
124   const osc = AC.createOscillator(), gain = AC.createGain(), t0 = AC.currentTime + delay;
125   osc.type = type;
```

Lines 124 to 125. An oscillator makes the raw wave and a gain node controls its volume. t0 is the moment it should start.

```
126   osc.frequency.setValueAtTime(f0, t0);
127   osc.frequency.exponentialRampToValueAtTime(Math.max(1, f1), t0 + dur);
```

Lines 126 to 127. Slide the pitch smoothly from f0 to f1 over the length of the sound.

```
128   gain.gain.setValueAtTime(0.0001, t0);
129   gain.gain.exponentialRampToValueAtTime(vol, t0 + 0.01);
130   gain.gain.exponentialRampToValueAtTime(0.0001, t0 + dur);
```

Lines 128 to 130. Shape the volume: fade in very fast (0.01 seconds, which avoids a click), then fade out for the rest of the sound.

```
131   osc.connect(gain).connect(AC.destination);
132   osc.start(t0); osc.stop(t0 + dur + 0.05);
133 }
```

Lines 131 to 133. Wire oscillator to volume control to speakers, then start it and schedule it to stop.

```
134 function volAt(x, y) { return Math.max(0, 1 - Math.hypot(x - player.x, y - player.y) / 600); }
```

Line 134. volAt() works out how loud something at (x, y) should sound to you: full volume right next to you, fading to silent 600 pixels away.

```
const sfx = {
```

Line 135. The game's sound effects. Each one is built from one or more tones.

```
  ping() { tone(1400, 700, 0.3, 0.22); tone(1300, 650, 0.3, 0.07, 'sine', 0.2); tone(1200, 600, 0.3, 0.025, 'sine', 0.4); },
```

Line 136. Sonar: a high falling ping, then two quieter repeats that sound like an echo.

```
  click() { tone(2600, 1800, 0.05, 0.09, 'square'); tone(2400, 1700, 0.05, 0.02, 'square', 0.09); },
```

Line 137. Quiet click: a very short, high, buzzy tick.

```
  step() { tone(140, 60, 0.08, 0.14, 'triangle'); },
```

Line 138. Your running footstep: a soft low thump.

```
  thud(v) { tone(70, 35, 0.14, 0.3 * v, 'triangle'); },
```

Line 139. A creature's footstep: deeper, and quieter the further away it is (v is the volume from volAt).

```
  growl(v) { tone(80, 38, 0.7, 0.3 * v, 'sawtooth'); },
```

Line 140. The growl a creature makes when it hears you.

```
  beat(v) { tone(65, 40, 0.12, 0.35 * v); tone(60, 38, 0.12, 0.22 * v, 'sine', 0.16); },
```

Line 141. Your heartbeat: a double "lub-dub" that gets louder as danger gets closer.

```
  win() { [523, 659, 784, 1047].forEach((f, i) => tone(f, f, 0.35, 0.12, 'triangle', i * 0.1)); },
```

Line 142. A rising four-note tune when you reach an exit. Each note starts 0.1 seconds after the last.

```
  chime() { tone(880, 880, 0.15, 0.08, 'triangle'); tone(1320, 1320, 0.2, 0.08, 'triangle', 0.1); },
```

Line 143. A short two-note chime when you finish a tutorial step.

```
  die() { tone(320, 30, 1.2, 0.3, 'sawtooth'); tone(90, 30, 1.2, 0.3, 'square'); }
};
```

Lines 144 to 145. A harsh falling crash when you are caught.

8. Game state

```
147 let state = 'title';
```

Line 147. Which screen we are on: 'title', 'play', 'dead' (just caught), 'won' (just escaped) or 'over' (no lives left).

```
148 let tutorial = false, level = 1, lives = MAX_LIVES, best = 1, seedSalt = 1;
```

Line 148. tutorial is true while the tutorial is running. level and lives track progress. best is the highest level you have reached. seedSalt is the random seed for the current maze.

```
149 let t = 0, stateT = 0;
```

Line 149. t is the game clock in seconds. stateT counts down the short pause after being caught or escaping.

```
150 let player = { x: 0, y: 0, r: 9 }, enemies = [], exitT = { tx: -1, ty: -1 };
```

Line 150. You (position x, y and size r), the list of creatures, and the exit tile.

```
151 let parts = [], pending = [], rings = [], hearQ = [];
```

Line 151. Lists of visual effects. pending holds echoes still travelling back to you. parts holds echoes glowing on screen now. rings holds expanding circles (sonar, noise, footsteps). hearQ holds noises still travelling towards creatures.

```
152 let msg = '', sub = '', msgT = 0;
```

Line 152. msg is the big message in the middle of the screen, sub is the smaller line under it, and msgT is how many more seconds they show.

```
153 let cool = 0, cool2 = 0, stepT = 0, hbT = 0, shake = 0, flash = 0;
```

Line 153. cool and cool2 are recharge timers for the pulse and the click. stepT times your running footsteps and hbT your heartbeat. shake and flash control the screen shake and the red flash.

```
154 try { best = +localStorage.getItem('echo_best') || 1; } catch (e) { }
```

Line 154. Load your best level from the browser's saved storage. The try/catch protects against browsers that block storage, such as some private windows.

```
155 function saveBest(n) {
156   if (n <= best) return;
157   best = n;
158   try { localStorage.setItem('echo_best', best); } catch (e) { }
159 }
```

Lines 155 to 159. saveBest() records a new best level (only if it beats the old one), both in memory and in saved storage.

9. Starting a game and a level

```
161 function clearFx() {
162   parts = []; pending = []; rings = []; hearQ = [];
163   cool = 0; cool2 = 0; flash = 0; shake = 0;
164 }
```

Lines 161 to 164. clearFx() empties every effect list and resets the timers. It runs whenever a level starts or restarts.

```
165 function makeEnemy(tx, ty) {
166   return { x: mid(tx), y: mid(ty), r: 11, path: [], mode: 'wander', idle: 1 + R() * 2, stepT: R(), alert:
    0, target: null, cause: '' };
167 }
```

Lines 165 to 167. makeEnemy() creates a creature in the centre of tile (tx, ty). r is its size. path is the list of tiles it is walking along. mode is 'wander', 'hunt' or 'search'. idle is how long it waits before wandering again. stepT times its footsteps. alert is how long it stays lit up after hearing something. target is where it heard the noise, and cause is which kind of noise it was, so the game can tell you why you were caught.

```
168 function startGame() {
169   tutorial = false; level = 1; lives = MAX_LIVES;
170   startLevel(true);
171 }
```

Lines 168 to 171. startGame() begins a fresh run: the real game, level 1, full lives, a new maze.

```
172 function startLevel(newMaze) {
173   if (newMaze) seedSalt = Math.floor(Math.random() * 1e9);
```

Lines 172 to 173. startLevel() builds and starts the current level. If newMaze is true it picks a new random seed. If false (after losing a life) it keeps the old seed, so you get the same maze again.

```
174   R = makeRng(level * 7919 + seedSalt);
```

Line 174. Create this level's random generator from the level number and the seed.

```
175   const n = Math.min(41, 13 + level * 4);
```

Line 175. The maze grows by 4 tiles each level, up to 41 by 41. 13 + 4 times anything is always odd, which the maze method needs.

```
176   genMaze(n, n, Math.floor(n * n * 0.035));
```

Line 176. Build the maze, with extra loop openings equal to about 3.5 percent of its area.

```
177   player = { x: mid(1), y: mid(1), r: 9 };
```

Line 177. Put you in the top-left floor tile.

```
178   const { dist } = bfs(1, 1);
```

Line 178. Flood out from you to find how many steps away every tile is.

```
179   let far = 0;
180   for (let i = 0; i < dist.length; i++) if (dist[i] > dist[far]) far = i;
181   exitT = { tx: far % GW, ty: (far / GW) | 0 };
```

Lines 179 to 181. The exit goes on the tile furthest from you, so you always have to cross the maze.

```
182   let spots = openTiles.filter(i => dist[i] >= 12 && i !== far);
183   if (!spots.length) spots = openTiles.filter(i => dist[i] >= 4 && i !== far);
```

Lines 182 to 183. Creatures may only start at least 12 steps from you and never on the exit. Tiny mazes fall back to 4 steps.

```
184   enemies = [];
185   const count = Math.min(level + 1, 9);
186   for (let k = 0; k < count && spots.length; k++) {
187     const i = spots.splice(Math.floor(R() * spots.length), 1)[0];
188     enemies.push(makeEnemy(i % GW, (i / GW) | 0));
189   }
```

Lines 184 to 189. Place one more creature than the level number, up to 9. Each chosen spot is removed from the list (splice) so two creatures never share one.

```
190   clearFx();
191   state = 'play';
192   msg = 'LEVEL ' + level;
193   sub = level === 1 ? 'Find the gold exit. Walking is silent, noise is not.' : '';
194   msgT = 2.5;
195 }
```

Lines 190 to 195. Reset effects, switch to playing and show the level banner. Level 1 also reminds you of the key rule.

10. The tutorial

```
197 const TUT_MAP = [
198   '########################',
199   '#.....#..............#',
200   '#.....#..............#',
201   '#..P..#.........C.....#',
202   '#....................#',
203   '#.....#..............#',
204   '###################.#####',
205   '#X.................#####',
206   '########################'
207 ];
```

Lines 197 to 207. The tutorial map, drawn as text. # is wall, . is floor, P is where you start, C is the creature and X is the exit. You start in the small room on the left, the creature waits in the big room on the right, and the exit is at the far end of the corridor along the bottom.

```
208 let tut = { pulses: 0, clicks: 0, run: 0, t: 0, step: 0 };
```

Line 208. tut keeps count of what you have done in the tutorial: pulses, clicks, seconds spent running, seconds on the current step, and which step you are on.

```
209 const TUT_STEPS = [
```

Line 209. The tutorial steps. Each has the instruction to show and a done() check. As soon as done() is true, the tutorial moves on to the next step.

```
210   { text: 'It is pitch black. Press SPACE (or tap) to send a sonar pulse and see what is around you.',
211     done: () => tut.pulses > 0 },
```

Lines 210 to 211. Step 1: send your first pulse.

```
212   { text: 'Cyan echoes are walls. They fade, so pulse again when you need to. Walk with WASD or the arrow keys (or drag) through the gap on the right.',
213     done: () => tileOf(player.x) >= 7 },
```

Lines 212 to 213. Step 2: walk into the next room. Tile column 7 is the first one past the gap in the wall.

```
214   { text: 'A creature lives in this room. It is blind too and hunts by SOUND. The dashed red ring shows how far your noise travels. Pulse now and watch.',
215     done: () => enemies[0].mode === 'hunt' },
```

Lines 214 to 215. Step 3: make a noise close enough for the creature to hear, so it starts hunting.

```
216   { text: 'The red X is where it heard you. It walks to THAT spot, not to you. Walking makes no sound, so just move away. (It cannot hurt you in the tutorial.)',
217     done: () => enemies[0].mode !== 'hunt' && tut.t > 1 },
```

Lines 216 to 217. Step 4: let it reach the spot it heard and stop hunting. The one-second wait gives you time to read.

```
218   { text: 'Press E (or tap with two fingers) for a quiet click. It only shows walls close to you, and creatures only hear it from very near.',
219     done: () => tut.clicks > 0 },
```

Lines 218 to 219. Step 5: try the quiet click.

```
220   { text: 'Hold SHIFT to run (or drag far). Running is fast, but every footstep is a noise. Watch the sma
    ll red rings.',
221     done: () => tut.run > 1 },
```

Lines 220 to 221. Step 6: run for at least one second in total.

```
222   { text: 'Last step: find the gold exit. Hint: there is a gap in the bottom wall. In the real game you g
    et 3 lives.',
223     done: () => false }
224 ];
```

Lines 222 to 224. Step 7: find the exit. This check never passes by itself; reaching the exit ends the tutorial instead.

```
225 function startTutorial() {
226   tutorial = true; level = 1; lives = MAX_LIVES;
227   tut = { pulses: 0, clicks: 0, run: 0, t: 0, step: 0 };
```

Lines 225 to 227. startTutorial() loads the tutorial map and puts you, the creature and the exit on their marked spots.

```
228   R = makeRng(42);
229   const m = loadMap(TUT_MAP);
230   player = { x: mid(m.P.tx), y: mid(m.P.ty), r: 9 };
231   exitT = m.X;
```

Lines 228 to 231. A fixed seed so the tutorial creature always behaves the same way at the start.

```
232   const e = makeEnemy(m.C.tx, m.C.ty);
233   e.harmless = true; e.harmT = 0;
234   enemies = [e];
235   clearFx();
236   state = 'play';
237   msg = 'TUTORIAL'; sub = 'Press ESC or tap SKIP to jump to the real game.'; msgT = 3;
238 }
```

Lines 232 to 238. The tutorial creature is marked harmless. harmT is a short timer that stops it "catching" you over and over.

```
239 function updateTutorial(dt) {
240   tut.t += dt;
241   const step = TUT_STEPS[tut.step];
242   if (step && step.done()) { tut.step++; tut.t = 0; sfx.chime(); }
243 }
```

Lines 239 to 243. updateTutorial() runs every frame during the tutorial. It counts time on the current step, and when that step's check passes it moves on and plays a chime.

11. Echolocation (how you see)

```
245 function cast(ox, oy, ang, maxD) {
```

Line 245. cast() follows one straight ray from (ox, oy) at angle ang until it hits something or has gone maxD pixels. It uses the DDA method: instead of creeping forward a pixel at a time, it jumps straight from one grid line to the next, so it is fast and can never slip past a wall.

```
246   const dx = Math.cos(ang), dy = Math.sin(ang);
```

Line 246. The ray's direction, split into an x part and a y part.

```
247   let tx = tileOf(ox), ty = tileOf(oy);
```

Line 247. The tile the ray starts in.

```
248   const sx = dx > 0 ? 1 : -1, sy = dy > 0 ? 1 : -1;
```

Line 248. Which way to step through the tiles: +1 or -1 on each axis.

```
249   const stepX = dx === 0 ? Infinity : Math.abs(TILE / dx);
250   const stepY = dy === 0 ? Infinity : Math.abs(TILE / dy);
```

Lines 249 to 250. How far along the ray you travel to cross one whole tile sideways (stepX) or up/down (stepY). A ray that is perfectly flat never crosses, hence Infinity.

```
251   let nextX = dx === 0 ? Infinity : (dx > 0 ? ((tx + 1) * TILE - ox) / dx : (ox - tx * TILE) / -dx);
252   let nextY = dy === 0 ? Infinity : (dy > 0 ? ((ty + 1) * TILE - oy) / dy : (oy - ty * TILE) / -dy);
```

Lines 251 to 252. How far along the ray the first vertical grid line (nextX) and the first horizontal grid line (nextY) are.

```
253   let d = 0;
254   while (d < maxD) {
```

Lines 253 to 254. d is how far the ray has travelled so far.

```
255     if (nextX < nextY) { d = nextX; nextX += stepX; tx += sx; }
256     else { d = nextY; nextY += stepY; ty += sy; }
```

Lines 255 to 256. Cross whichever grid line comes first, and move into the next tile that way.

```
257     if (d > maxD) break;
```

Line 257. Stop if that took us past the range.

```
258     if (tx === exitT.tx && ty === exitT.ty) return { d, kind: EXIT };
259     if (solid(tx, ty)) return { d, kind: WALL };
260   }
```

Lines 258 to 260. Report the exit or a wall, along with how far away it was.

```
261   return null;
262 }
```

Lines 261 to 262. Nothing within range.

```
function sonar(ox, oy, range, rays, dim) {
```

Line 263. sonar() sends rays out in every direction from (ox, oy) and turns each hit into an echo dot. dim is true for quiet sounds, which draw fainter. This function only affects what YOU see. Whether creatures hear it is handled separately by makeNoise().

```
  rings.push({ x: ox, y: oy, t0: t, max: range, speed: SOUND, col: COL[WALL], a: dim ? 0.12 : 0.3, w: dim ? 1 : 2 });
```

Line 264. Add a cyan expanding circle that shows the sound wave spreading out.

```
  let prev = null;
  const off = Math.random() * 0.05;
```

Lines 265 to 266. prev remembers the previous echo so neighbouring echoes can be joined with a line. off is a tiny random twist so the dots do not land in exactly the same places every time.

```
  for (let i = 0; i <= rays; i++) {
    const a = (i % rays) / rays * Math.PI * 2 + off;
    const c = Math.cos(a), s = Math.sin(a);
```

Lines 267 to 269. One ray for each slice of the full circle. The loop runs one extra time so the last dot can join back to the first.

```
    const hit = cast(ox, oy, a, range);
    let d = hit ? hit.d : range, kind = hit ? hit.kind : 0;
```

Lines 270 to 271. Find what this ray hits. If nothing, d is the full range and kind is 0.

```
    for (const e of enemies) {
```

Line 272. Check whether a creature is in the way and closer than the wall. This is the standard ray-versus-circle test.

```
      const ex = e.x - ox, ey = e.y - oy, proj = ex * c + ey * s;
      if (proj > 0 && proj < d) {
```

Lines 273 to 274. proj is how far along the ray the creature's centre lies.

```
        const perp = Math.abs(ex * s - ey * c);
        if (perp < e.r) { d = proj - Math.sqrt(e.r * e.r - perp * perp); kind = FOE; }
      }
    }
```

Lines 275 to 278. perp is how far to the side of the ray the centre is. If that is less than the creature's size, the ray hits it, and Pythagoras gives the exact point on its edge.

```
    if (!kind) { prev = null; continue; }
```

Line 279. A ray that hits nothing makes no echo and breaks the joining line.

```
280      const p = {
281        x: ox + c * d, y: oy + s * d, at: t + d / SOUND,
282        kind: dim && kind === WALL ? DIMWALL : kind,
283        fade: (dim ? 0.45 : 1) * (1 - (d / range) * 0.6),
284        life: kind === FOE ? 1.6 : kind === EXIT ? 5 : 4.5
285      };
```

Lines 280 to 285. Build the echo. It sits where the ray hit. "at" is when it appears: now, plus the time sound takes to travel there. Quiet sounds use the dim wall colour. fade makes far echoes fainter. life is how long it glows; creatures fade fastest because they move.

```
286      if (prev && prev.kind === p.kind && Math.hypot(prev.x - p.x, prev.y - p.y) < 26) { p.lx = prev.x; p.l
     y = prev.y; }
```

Line 286. If the previous echo was the same kind and close by, store its position so a line can join the two. This turns dots into wall outlines.

```
287      pending.push(p);
288      prev = p;
289    }
290  }
```

Lines 287 to 290. Queue the echo to appear when its time comes, and remember it for the next ray.

12. Noise (how creatures find you)

```
292 function makeNoise(x, y, radius, cause) {
```

Line 292. makeNoise() holds the main stealth rule. Creatures cannot see. They only know where a sound came FROM. Every creature within the radius walks to that exact spot, so if you move away after making a noise, they arrive at an empty place.

```
293   rings.push({ x, y, t0: t, max: radius, speed: SOUND, col: RED, a: 0.4, w: 1.5, dash: true });
```

Line 293. Draw a dashed red ring that spreads at the speed of sound out to the hearing radius, so you can see exactly how far your noise carries.

```
294   for (const e of enemies) {
295     const d = Math.hypot(e.x - x, e.y - y);
296     if (d <= radius) hearQ.push({ e, at: t + d / SOUND, x, y, cause });
297   }
298 }
```

Lines 294 to 298. For every creature inside the radius, queue the moment the sound reaches it (distance divided by the speed of sound). So a creature reacts when the red ring touches it, not before.

```
299 function hear(h) {
300   const e = h.e;
```

Lines 299 to 300. hear() runs when a queued sound reaches a creature.

```
301   if (e.mode !== 'hunt') {
302     const v = volAt(e.x, e.y);
303     sfx.growl(v);
304     if (v > 0.3) shake = Math.max(shake, 6 * v);
305   }
```

Lines 301 to 305. If it was not already hunting, it growls (louder if it is near you), and if it is close the screen shakes a little.

```
306   e.alert = 1.5;
```

Line 306. Light it up for 1.5 seconds so you can see where it was when it heard you.

```
307   e.mode = 'hunt'; e.cause = h.cause; e.target = { x: h.x, y: h.y };
308   e.path = findPath(tileOf(e.x), tileOf(e.y), tileOf(h.x), tileOf(h.y));
```

Lines 307 to 308. Switch to hunting, remember what it heard and where, and work out its route to that spot.

```
309   if (!e.path.length) { e.mode = 'search'; e.idle = 2.5; }
310 }
```

Lines 309 to 310. If it is already standing on that spot, it skips straight to searching.

```
311 function emit(kind) {
312   sonar(player.x, player.y, SIGHT[kind], RAYS[kind], kind !== 'pulse');
313   makeNoise(player.x, player.y, HEAR[kind], kind);
314   if (tutorial && kind === 'pulse') tut.pulses++;
315   if (tutorial && kind === 'click') tut.clicks++;
316 }
```

Lines 311 to 316. emit() is what happens whenever you make a sound of a given kind ('pulse', 'click' or 'step'): you see by it (sonar) and creatures may hear it (makeNoise). The tutorial counts your pulses and clicks here too.

13. Controls

```
318 const keys = {};
```

Line 318. keys remembers which keys are being held down right now.

```
319 addEventListener('keydown', e => {
320   keys[e.code] = true;
321   if (['Space', 'ArrowUp', 'ArrowDown', 'ArrowLeft', 'ArrowRight'].includes(e.code)) e.preventDefault();
322   if (!e.repeat) onKey(e.code);
323 });
```

Lines 319 to 323. When a key goes down: remember it, stop space and the arrow keys from scrolling the page, and pass it to onKey(). Holding a key fires automatic repeats (e.repeat), which are ignored for actions.

```
324 addEventListener('keyup', e => { keys[e.code] = false; });
```

Line 324. When a key comes back up, forget it.

```
325 function onKey(code) {
326   initAudio();
```

Lines 325 to 326. onKey() decides what a key press means on each screen.

```
327   if (state === 'title') {
328     if (code === 'Space' || code === 'Enter') startGame();
329     if (code === 'KeyT') startTutorial();
330     return;
331   }
```

Lines 327 to 331. On the title screen: Space or Enter plays, T opens the tutorial.

```
332   if (state === 'over') {
333     if (code === 'Space' || code === 'Enter') startGame();
334     if (code === 'Escape') state = 'title';
335     return;
336   }
```

Lines 332 to 336. On the game over screen: Space or Enter tries again, Escape goes back to the title.

```
337   if (code === 'Escape' && tutorial) { startGame(); return; }
```

Line 337. During the tutorial, Escape skips straight to the real game.

```
338   if (code === 'Space') action('pulse');
339   if (code === 'KeyE') action('click');
340 }
```

Lines 338 to 340. While playing, Space pulses and E clicks.

```
341 let touch = null, joy = { x: 0, y: 0 };
```

Line 341. Touch controls. touch remembers where and when the finger went down. joy is a virtual joystick: which way and how far you have dragged, from -1 to 1 on each axis.

```
cv.addEventListener('touchstart', e => {
  e.preventDefault();
  initAudio();
  if (e.touches.length === 2) { action('click'); return; }
  const p = e.touches[0];
  touch = { x: p.clientX, y: p.clientY, t: performance.now(), moved: false };
}, { passive: false });
```

Lines 342 to 348. When a finger touches: two fingers means a quiet click. Otherwise remember the start point and time. preventDefault stops the browser scrolling or zooming.

```
cv.addEventListener('touchmove', e => {
  e.preventDefault();
  if (!touch) return;
  const p = e.touches[0], dx = p.clientX - touch.x, dy = p.clientY - touch.y, m = Math.hypot(dx, dy);
  if (m > 12) touch.moved = true;
  joy = m > 12 ? { x: dx / Math.max(m, 60), y: dy / Math.max(m, 60) } : { x: 0, y: 0 };
}, { passive: false });
```

Lines 349 to 355. When the finger moves: measure how far it is from where it started. Past 12 pixels it counts as a drag. The joystick points in the drag direction and reaches full strength at 60 pixels.

```
cv.addEventListener('touchend', e => {
  e.preventDefault();
  if (touch && !touch.moved && performance.now() - touch.t < 250) tap(touch.x, touch.y);
  if (!e.touches.length) { touch = null; joy = { x: 0, y: 0 }; }
}, { passive: false });
```

Lines 356 to 360. When the finger lifts: a quick touch (under a quarter of a second) without dragging counts as a tap. Once no fingers are left, reset the joystick.

```
cv.addEventListener('mousedown', e => { initAudio(); tap(e.clientX, e.clientY); });
```

Line 361. A mouse click works just like a tap.

```
let buttons = [];
```

Line 362. buttons holds the on-screen buttons drawn in the latest frame, so taps can be matched against them.

```
function tap(x, y) {
  for (const b of buttons) {
    if (x >= b.x && x <= b.x + b.w && y >= b.y && y <= b.y + b.h) { b.act(); return; }
  }
  if (state === 'play') action('pulse');
}
```

Lines 363 to 368. tap() first checks whether you tapped inside a button and, if so, runs it. Otherwise, while playing, a tap is a sonar pulse.

```
function action(kind) {
  if (state !== 'play') return;
  if (kind === 'pulse' && cool <= 0) { cool = 1.2; sfx.ping(); emit('pulse'); }
  if (kind === 'click' && cool2 <= 0) { cool2 = 0.35; sfx.click(); emit('click'); }
}
```

Lines 369 to 373. action() fires a pulse or a click, but only while playing and only once its recharge timer has run out. The pulse needs 1.2 seconds to recharge, the click 0.35 seconds.

14. Updating the world each frame

```
375 function hitWall(o) {
376   const r = o.r;
377   return solid(tileOf(o.x - r), tileOf(o.y - r)) || solid(tileOf(o.x + r), tileOf(o.y - r)) ||
378     solid(tileOf(o.x - r), tileOf(o.y + r)) || solid(tileOf(o.x + r), tileOf(o.y + r));
379 }
```

Lines 375 to 379. hitWall() checks the four corners of a round object's bounding box. If any corner is inside a wall tile, the object is touching a wall.

```
380 function update(dt) {
```

Line 380. update() moves the whole game forward by dt seconds. It runs about 60 times a second.

```
381   for (let i = pending.length - 1; i >= 0; i--) {
382     if (pending[i].at <= t) { const p = pending[i]; p.born = t; parts.push(p); pending.splice(i, 1); }
383   }
```

Lines 381 to 383. Move echoes from "still travelling" to "visible" once their arrival time has passed. The loop runs backwards so removing items does not skip any.

```
384   parts = parts.filter(p => t - p.born < p.life);
385   rings = rings.filter(r => (t - r.t0) * r.speed < r.max);
```

Lines 384 to 385. Throw away echoes that have faded out and rings that have finished spreading.

```
386   msgT -= dt; shake *= 0.9; flash = Math.max(0, flash - dt);
```

Line 386. Count down the message, calm the screen shake and fade the red flash.

```
387   if (state === 'dead' || state === 'won') {
388     stateT -= dt;
389     if (stateT <= 0) afterPause();
390     return;
391   }
```

Lines 387 to 391. During the short pause after being caught or escaping, just count down, then move on.

```
392   if (state !== 'play') return;
```

Line 392. Nothing else moves unless we are playing.

```
393   cool -= dt; cool2 -= dt;
```

Line 393. Recharge the pulse and the click.

```
394   for (let i = hearQ.length - 1; i >= 0; i--) {
395     if (hearQ[i].at <= t) { hear(hearQ[i]); hearQ.splice(i, 1); }
396   }
```

Lines 394 to 396. Deliver any sounds that have now reached a creature.

```
397   movePlayer(dt);
398   moveEnemies(dt);
```

Lines 397 to 398. Move you, then the creatures.

```
399   if (state !== 'play') return;
```

Line 399. If a creature just caught you, the round is over, so stop here.

```
400   checkExit();
401   if (tutorial) updateTutorial(dt);
402 }
```

Lines 400 to 402. Check for the exit, and run the tutorial's step checks if it is active.

```
403 function movePlayer(dt) {
```

Line 403. movePlayer() reads the controls and moves you.

```
404   const mx = (keys.KeyD || keys.ArrowRight ? 1 : 0) - (keys.KeyA || keys.ArrowLeft ? 1 : 0) + joy.x;
405   const my = (keys.KeyS || keys.ArrowDown ? 1 : 0) - (keys.KeyW || keys.ArrowUp ? 1 : 0) + joy.y;
406   const m = Math.hypot(mx, my);
```

Lines 404 to 406. Add up the direction from the keys (right minus left, down minus up) plus the touch joystick.

```
407   const running = keys.ShiftLeft || keys.ShiftRight || Math.hypot(joy.x, joy.y) > 0.95;
```

Line 407. You are running if Shift is held or the touch drag is at full stretch.

```
408   if (m < 0.05) return;
```

Line 408. No real input means stand still.

```
409   const sp = (running ? RUN : WALK) * dt / Math.max(1, m);
```

Line 409. How far to move this frame. Dividing by m stops diagonal movement being faster than straight movement.

```
410   player.x += mx * sp; if (hitWall(player)) player.x -= mx * sp;
411   player.y += my * sp; if (hitWall(player)) player.y -= my * sp;
```

Lines 410 to 411. Move sideways and undo it if that put you in a wall. Then the same for up and down. Doing the two separately lets you slide along walls instead of sticking to them.

```
412   if (running) {
413     if (tutorial) tut.run += dt;
414     stepT -= dt;
415     if (stepT <= 0) { stepT = 0.3; sfx.step(); emit('step'); }
416   }
417 }
```

Lines 412 to 417. Running makes a footstep every 0.3 seconds. Each footstep is a tiny sound you can see by and a noise creatures can hear. The tutorial counts running time.

```
418 function moveEnemies(dt) {
```

Line 418. moveEnemies() runs each creature's simple brain and checks whether any of them has reached you.

```
419   const huntSpeed = tutorial ? 80 : Math.min(140, 92 + level * 6);
```

Line 419. Hunting speed rises a little each level, up to a cap. Early on, walking (115) is faster than hunting, so you can always walk away. The tutorial creature is slower still.

```
420   let nearest = Infinity;
421   for (const e of enemies) {
```

Lines 420 to 421. nearest tracks the closest creature, for the heartbeat and the red screen edge.

```
422     e.alert = Math.max(0, e.alert - dt);
423     if (e.harmT) e.harmT = Math.max(0, e.harmT - dt);
```

Lines 422 to 423. Count down how long it stays lit up, and its tutorial "just caught you" timer.

```
424     const d = Math.hypot(e.x - player.x, e.y - player.y);
```

Line 424. How far it is from you.

```
425     e.stepT -= dt;
```

Line 425. Creature footsteps, so you can tell something is coming in the dark.

```
426     if (e.stepT <= 0 && e.path.length) {
427       const hunting = e.mode === 'hunt';
428       e.stepT = hunting ? 0.45 : 1.3;
429       if (d < (hunting ? 650 : 300)) {
```

Lines 426 to 429. Only a moving creature makes footsteps. Hunting creatures stomp every 0.45 seconds and can be noticed up to 650 pixels away. Wandering ones shuffle every 1.3 seconds and are only noticed within 300 pixels.

```
430         rings.push({ x: e.x, y: e.y, t0: t, max: hunting ? 55 : 35, speed: 110, col: RED, a: hunting ? 0.
    55 : 0.3, w: 1.5 });
431         sfx.thud(volAt(e.x, e.y) * (hunting ? 1 : 0.5));
432       }
433     }
```

Lines 430 to 433. Each footstep is a small red ripple at the creature's feet plus a thud you can hear.

```
434     if (!e.path.length) {
435       e.idle -= dt;
```

Lines 434 to 435. With nowhere to go, it waits.

```
436       if (e.idle <= 0) {
437         e.mode = 'wander'; e.target = null; e.cause = '';
438         const ex = tileOf(e.x), ey = tileOf(e.y);
439         for (let k = 0; k < 12; k++) {
440           const i = openTiles[Math.floor(Math.random() * openTiles.length)];
441           const tx = i % GW, ty = (i / GW) | 0;
442           if (Math.abs(tx - ex) + Math.abs(ty - ey) < 9) { e.path = findPath(ex, ey, tx, ty); break; }
443         }
444         e.idle = 1 + Math.random() * 2;
445       }
446     } else {
```

Lines 436 to 446. When the wait is over it calms down, forgets the last noise, and picks a random floor tile within 9 steps to wander to (trying up to 12 times to find one that close).

```
447       const n = e.path[0], px = mid(n % GW), py = mid((n / GW) | 0);
448       const dx = px - e.x, dy = py - e.y, dd = Math.hypot(dx, dy);
449       const sp = (e.mode === 'hunt' ? huntSpeed : 45) * dt;
```

Lines 447 to 449. Otherwise it heads for the centre of the next tile on its path. Hunting is fast, wandering is a slow 45 pixels per second.

```
450       if (dd <= sp) {
451         e.x = px; e.y = py; e.path.shift();
452         if (!e.path.length && e.mode === 'hunt') { e.mode = 'search'; e.idle = 2.5; }
453       } else {
```

Lines 450 to 453. If it can reach that tile this frame, it snaps onto it and moves on to the next. When a hunt path runs out it has reached the noise: it switches to 'search' and sniffs around for 2.5 seconds before giving up.

```
454         e.x += dx / dd * sp; e.y += dy / dd * sp;
455       }
456     }
```

Lines 454 to 456. Otherwise it takes a step of the right length straight towards the tile.

```
457     nearest = Math.min(nearest, d);
```

Line 457. Keep track of the closest creature.

```
458     if (d < e.r + player.r - 2) {
459       if (e.harmless) { if (!e.harmT) tutorialCaught(e); }
460       else { caught(e); return; }
461     }
462   }
```

Lines 458 to 462. Touching means the two circles overlap (with 2 pixels of forgiveness). In the tutorial nothing bad happens. In the real game you lose a life.

```
463   player.danger = Math.max(0, 1 - nearest / 260);
```

Line 463. The red edge of the screen grows as a creature comes within 260 pixels.

```
464   if (nearest < 320) {
465     hbT -= dt;
466     if (hbT <= 0) { sfx.beat(1 - nearest / 320); hbT = 0.32 + (nearest / 320) * 0.9; }
467   }
468 }
```

Lines 464 to 468. Within 320 pixels your heartbeat starts, getting faster and louder the closer the creature is.

```
469 function caught(e) {
```

Line 469. caught() runs when a creature reaches you in the real game.

```
470   lives--;
471   state = 'dead'; stateT = 2.4; flash = 1; shake = 18; sfx.die();
```

Lines 470 to 471. Lose a life, start the pause, flash red, shake hard and play the crash.

```
472   msg = { pulse: 'IT HEARD YOUR SONAR', step: 'IT HEARD YOU RUNNING', click: 'IT HEARD YOUR CLICK' }[e.ca
    use] || 'IT BUMPED INTO YOU';
```

Line 472. Tell you WHY it found you, based on the last noise it heard. If it heard nothing, it simply bumped into you while wandering.

```
473   sub = lives > 0 ? lives + (lives === 1 ? ' life' : ' lives') + ' left' : 'No lives left';
474   msgT = 2.4;
```

Lines 473 to 474. Say how many lives are left. Note the singular "life" when only one remains.

```
475   for (const o of enemies) o.alert = 2.4;
476   rings.push({ x: player.x, y: player.y, t0: t, max: 400, speed: SOUND, col: RED, a: 0.5, w: 2 });
477 }
```

Lines 475 to 477. Light up every creature so you can see where they all were, and send out a red ring from where you fell.

```
478 function tutorialCaught(e) {
479   flash = 0.6; shake = 8;
480   msg = 'CAUGHT'; sub = 'In the real game that costs a life. Keep going.'; msgT = 2.5;
481   e.harmT = 2; e.mode = 'wander'; e.path = []; e.idle = 2; e.target = null;
482 }
```

Lines 478 to 482. tutorialCaught() is the friendly tutorial version: a red flash and a message, and the creature wanders off. harmT stops it happening again for 2 seconds.

```
483 function checkExit() {
484   if (tileOf(player.x) !== exitT.tx || tileOf(player.y) !== exitT.ty) return;
```

Lines 483 to 484. checkExit() looks at whether you are standing on the exit tile, and stops early if not.

```
485   state = 'won'; stateT = tutorial ? 2.8 : 1.8; sfx.win();
486   rings.push({ x: player.x, y: player.y, t0: t, max: 600, speed: SOUND, col: COL[EXIT], a: 0.5, w: 2 });
```

Lines 485 to 486. Escaped! Start the pause, play the tune and send out a gold ring.

```
487   if (tutorial) { msg = 'TUTORIAL DONE'; sub = 'Now for real. 3 lives. Good luck.'; }
488   else { msg = 'ESCAPED'; sub = ''; saveBest(level + 1); }
489   msgT = stateT;
490 }
```

Lines 487 to 490. Different messages for finishing the tutorial and finishing a level. Reaching a new level also updates your best.

```
491 function afterPause() {
```

Line 491. afterPause() decides what happens once the pause is over.

```
492   if (state === 'won') {
493     if (tutorial) startGame();
494     else { level++; startLevel(true); }
```

Lines 492 to 494. After escaping: the tutorial leads into the real game, and a real level leads to the next level with a new maze.

```
495   } else if (lives > 0) {
496     startLevel(false);
497   } else {
498     state = 'over';
499   }
500 }
```

Lines 495 to 500. After being caught: with lives left you retry the same maze. With none left it is game over.

15. Drawing the world

```
502 function drawRings() {
503   for (const r of rings) {
504     const rad = (t - r.t0) * r.speed;
505     cx.globalAlpha = r.a * (1 - rad / r.max);
506     cx.strokeStyle = r.col; cx.lineWidth = r.w;
```

Lines 502 to 506. drawRings() draws every expanding circle. Its radius is the time since it began multiplied by its speed, and it fades as it grows.

```
507     cx.setLineDash(r.dash ? [6, 6] : []);
508     cx.beginPath(); cx.arc(r.x, r.y, rad, 0, Math.PI * 2); cx.stroke();
509   }
```

Lines 507 to 509. Noise rings are dashed so they look different from sonar rings. An empty list means a solid line.

```
510   cx.setLineDash([]);
511 }
```

Lines 510 to 511. Switch dashes off again so nothing else is drawn dashed by accident.

```
512 function drawEchoes() {
513   cx.lineWidth = 1.2;
514   for (const p of parts) {
```

Lines 512 to 514. drawEchoes() draws every visible echo as a small square, plus a line back to its neighbour where it has one.

```
515     let a = p.fade * Math.pow(1 - (t - p.born) / p.life, 1.6);
516     if (p.kind === EXIT) a *= 0.7 + 0.3 * Math.sin(t * 8 + p.x);
```

Lines 515 to 516. Brightness falls away as the echo ages (the power of 1.6 makes it linger and then drop off). Exit echoes also shimmer.

```
517     if (a <= 0.01) continue;
518     cx.globalAlpha = a;
519     cx.fillStyle = cx.strokeStyle = COL[p.kind];
```

Lines 517 to 519. Skip anything too faint to see.

```
520     if (p.lx !== undefined) { cx.beginPath(); cx.moveTo(p.lx, p.ly); cx.lineTo(p.x, p.y); cx.stroke(); }
521     const s = p.kind === FOE ? 3 : 2;
522     cx.fillRect(p.x - s / 2, p.y - s / 2, s, s);
523   }
524 }
```

Lines 520 to 524. Draw the joining line, then the dot itself. Creature dots are a bit bigger.

```
525 function drawCreatureHints() {
526   for (const e of enemies) {
```

Lines 525 to 526. drawCreatureHints() shows what creatures are doing, but only what they have given away by reacting to noise.

```
    if (e.mode === 'hunt' && e.target) {
      const { x, y } = e.target;
      cx.globalAlpha = 0.55 + 0.25 * Math.sin(t * 6);
      cx.strokeStyle = RED; cx.lineWidth = 2;
      cx.beginPath();
      cx.moveTo(x - 7, y - 7); cx.lineTo(x + 7, y + 7);
      cx.moveTo(x + 7, y - 7); cx.lineTo(x - 7, y + 7);
      cx.stroke();
    }
```

Lines 527 to 535. While it hunts, a pulsing red X marks the spot it is heading for: where it heard you.

```
    if (e.alert > 0) {
      const a = Math.min(1, e.alert);
      const g = cx.createRadialGradient(e.x, e.y, 0, e.x, e.y, 22);
      g.addColorStop(0, `rgba(255,60,60,${0.9 * a})`); g.addColorStop(1, 'rgba(255,0,0,0)');
      cx.globalAlpha = 1; cx.fillStyle = g;
      cx.beginPath(); cx.arc(e.x, e.y, 22, 0, Math.PI * 2); cx.fill();
```

Lines 536 to 541. Just after it hears something (or when you are caught) it glows red for a moment.

```
      cx.globalAlpha = a; cx.fillStyle = RED;
      cx.font = `bold 16px ${FONT}`; cx.textAlign = 'center';
      cx.fillText('!', e.x, e.y - 18);
    }
  }
}
```

Lines 542 to 547. ...with a "!" above its head.

```
function drawPlayer() {
  cx.globalAlpha = 1;
  const g = cx.createRadialGradient(player.x, player.y, 0, player.x, player.y, 26);
  g.addColorStop(0, 'rgba(200,255,255,0.9)');
  g.addColorStop(0.3, 'rgba(80,220,255,0.35)');
  g.addColorStop(1, 'rgba(0,0,0,0)');
  cx.fillStyle = g;
  cx.beginPath(); cx.arc(player.x, player.y, 26, 0, Math.PI * 2); cx.fill();
}
```

Lines 548 to 556. drawPlayer() draws you as a soft glowing dot: a radial gradient fading from bright cyan-white in the middle to clear at the edge.

```
function draw() {
```

Line 557. draw() paints one whole frame.

```
  cx.setTransform(DPR, 0, 0, DPR, 0, 0);
  cx.globalCompositeOperation = 'source-over'; cx.globalAlpha = 1;
  cx.fillStyle = '#000'; cx.fillRect(0, 0, VW, VH);
```

Lines 558 to 560. Scale everything by the pixel density so the rest of the code can think in normal CSS pixels. Then reset blending and clear the screen to black.

```
  buttons = [];
```

Line 561. Forget last frame's buttons. Whichever screen is drawn will add its own.

```
  if (state === 'title') { drawTitle(); return; }
  if (state === 'over') { drawGameOver(); return; }
```

Lines 562 to 563. The title and game over screens have their own drawing functions.

```
564   const sx = (Math.random() - 0.5) * shake, sy = (Math.random() - 0.5) * shake;
```

Line 564. A random offset for screen shake (zero when shake is zero).

```
565   cx.save();
566   cx.translate(Math.round(VW / 2 - player.x + sx), Math.round(VH / 2 - player.y + sy));
```

Lines 565 to 566. Move the "camera" so you are always in the centre of the screen. save() lets us undo this afterwards with restore().

```
567   cx.globalCompositeOperation = 'lighter';
```

Line 567. "lighter" blending adds overlapping glows together so they get brighter, like real light.

```
568   drawRings();
569   drawEchoes();
570   drawCreatureHints();
571   if (state !== 'dead') drawPlayer();
572   cx.restore();
```

Lines 568 to 572. Draw the world, back to front. You are hidden while the "caught" pause plays.

```
573   cx.globalCompositeOperation = 'source-over';
```

Line 573. Back to normal blending for the screen overlay.

```
574   const danger = state === 'play' ? (player.danger || 0) : 0;
575   if (danger > 0 || flash > 0) {
576     const v = Math.max(danger * (0.55 + 0.15 * Math.sin(t * 10)), flash);
```

Lines 574 to 576. A red vignette (glowing red edges) that pulses when a creature is near, or flashes when you are hit.

```
577     const g = cx.createRadialGradient(VW / 2, VH / 2, Math.min(VW, VH) * 0.25, VW / 2, VH / 2, Math.max(V
    W, VH) * 0.75);
578     g.addColorStop(0, 'rgba(255,0,30,0)');
579     g.addColorStop(1, `rgba(255,0,30,${v * 0.6})`);
580     cx.globalAlpha = 1; cx.fillStyle = g; cx.fillRect(0, 0, VW, VH);
581   }
```

Lines 577 to 581. The gradient is clear in the middle and red towards the edges of the screen.

```
582   drawHUD();
583 }
```

Lines 582 to 583. Finally the text and buttons on top.

16. Screen text and buttons (the HUD)

```
585 function heart(x, y, s, filled) {
586   cx.beginPath();
587   cx.moveTo(x, y + s * 0.3);
588   cx.bezierCurveTo(x, y, x - s / 2, y, x - s / 2, y + s * 0.3);
589   cx.bezierCurveTo(x - s / 2, y + s * 0.6, x, y + s * 0.8, x, y + s);
590   cx.bezierCurveTo(x, y + s * 0.8, x + s / 2, y + s * 0.6, x + s / 2, y + s * 0.3);
591   cx.bezierCurveTo(x + s / 2, y, x, y, x, y + s * 0.3);
592   if (filled) cx.fill(); else cx.stroke();
593 }
```

Lines 585 to 593. heart() draws a small heart at (x, y), s pixels tall, using four curves. Filled hearts are lives you still have, outlines are lives lost.

```
594 function wrap(text, maxW) {
595   const lines = []; let line = '';
596   for (const word of text.split(' ')) {
597     const test = line ? line + ' ' + word : word;
598     if (cx.measureText(test).width > maxW && line) { lines.push(line); line = word; }
599     else line = test;
600   }
601   if (line) lines.push(line);
602   return lines;
603 }
```

Lines 594 to 603. wrap() splits a long sentence into lines that fit within maxW pixels, adding one word at a time until the next word would not fit.

```
604 function box(x, y, w, h, r) {
605   cx.beginPath();
606   if (cx.roundRect) cx.roundRect(x, y, w, h, r); else cx.rect(x, y, w, h);
607 }
```

Lines 604 to 607. box() draws a rounded rectangle path, falling back to a plain rectangle on older browsers without roundRect.

```
608 function button(label, x, y, w, h, act, strong) {
609   cx.globalAlpha = 1;
610   cx.fillStyle = strong ? 'rgba(120,255,255,0.18)' : 'rgba(120,255,255,0.06)';
611   cx.strokeStyle = 'rgba(150,255,255,0.7)'; cx.lineWidth = 1.5;
612   box(x, y, w, h, 8); cx.fill(); cx.stroke();
```

Lines 608 to 612. button() draws a button with a label and remembers it so tap() can detect presses. act is the function to run when pressed. strong makes it slightly brighter, for the main choice.

```
613   cx.fillStyle = '#cff'; cx.font = `bold 15px ${FONT}`; cx.textAlign = 'center'; cx.textBaseline = 'middl
    e';
614   cx.fillText(label, x + w / 2, y + h / 2 + 1);
615   cx.textBaseline = 'alphabetic';
616   buttons.push({ x, y, w, h, act });
617 }
```

Lines 613 to 617. The label, centred in the button both ways.

```
618 function drawHUD() {
619   cx.globalAlpha = 1; cx.globalCompositeOperation = 'source-over';
620   cx.textAlign = 'left'; cx.font = `13px ${FONT}`; cx.fillStyle = 'rgba(150,255,255,0.75)';
```

Lines 618 to 620. drawHUD() draws everything that stays fixed on screen while you play.

```
621   cx.fillText(tutorial ? 'TUTORIAL' : `LEVEL ${level}   BEST ${best}`, 16, 26);
```

Line 621. Top left: the level (or TUTORIAL) and your best.

```
622   cx.fillStyle = cx.strokeStyle = RED; cx.lineWidth = 1.5;
623   for (let i = 0; i < MAX_LIVES; i++) heart(24 + i * 22, 38, 14, i < lives);
```

Lines 622 to 623. Your lives as red hearts.

```
624   const charge = 1 - Math.max(0, cool) / 1.2;
625   cx.fillStyle = 'rgba(150,255,255,0.15)'; cx.fillRect(16, 62, 120, 3);
626   cx.fillStyle = charge >= 1 ? 'rgba(150,255,255,0.9)' : 'rgba(150,255,255,0.45)';
627   cx.fillRect(16, 62, 120 * charge, 3);
```

Lines 624 to 627. The pulse recharge bar: a faint track, and a fill that grows back after each pulse and turns bright when ready.

```
628   cx.font = `11px ${FONT}`;
629   [[COL[WALL], 'wall'], [COL[EXIT], 'exit'], [RED, 'creature / noise']].forEach(([c, label], i) => {
630     cx.fillStyle = c; cx.fillRect(16, 76 + i * 15, 8, 8);
631     cx.fillStyle = 'rgba(170,255,255,0.6)'; cx.fillText(label, 30, 84 + i * 15);
632   });
```

Lines 628 to 632. A small colour key so you always know what each colour means.

```
633   if (msgT > 0) {
634     cx.globalAlpha = Math.min(1, msgT);
635     cx.textAlign = 'center';
636     cx.fillStyle = state === 'dead' || msg === 'CAUGHT' ? RED : state === 'won' ? COL[EXIT] : '#bff';
637     cx.font = `bold ${Math.min(28, VW / 16)}px ${FONT}`;
638     cx.fillText(msg, VW / 2, VH * 0.28);
```

Lines 633 to 638. The big centre message, fading out over its final second. Red for bad news, gold for escaping, cyan otherwise.

```
639     cx.font = `14px ${FONT}`; cx.fillStyle = 'rgba(200,255,255,0.85)';
640     if (sub) wrap(sub, VW - 32).forEach((l, i) => cx.fillText(l, VW / 2, VH * 0.28 + 28 + i * 18));
641     cx.globalAlpha = 1;
642   }
```

Lines 639 to 642. The smaller line underneath, wrapped so it fits on phones.

```
643   if (tutorial) drawTutorialPanel();
644 }
```

Lines 643 to 644. In the tutorial, also show the current instruction and the SKIP button.

```
645 function drawTutorialPanel() {
646   const step = TUT_STEPS[tut.step];
```

Lines 645 to 646. drawTutorialPanel() shows a box at the bottom of the screen with the current step, and a SKIP button in the top-right corner.

```
  const w = Math.min(VW - 32, 620);
  cx.font = `14px ${FONT}`;
  const lines = wrap(step.text, w - 32);
  const h = 44 + lines.length * 20;
  const x = (VW - w) / 2, y = VH - h - 16;
```

Lines 647 to 651. Work out the box size: up to 620 pixels wide, tall enough for the wrapped text, sitting 16 pixels above the bottom edge.

```
  cx.globalAlpha = 1;
  cx.fillStyle = 'rgba(0,20,24,0.85)'; cx.strokeStyle = 'rgba(150,255,255,0.4)'; cx.lineWidth = 1;
  box(x, y, w, h, 10); cx.fill(); cx.stroke();
```

Lines 652 to 654. The dark box with a thin cyan border.

```
  cx.textAlign = 'left';
  cx.fillStyle = COL[EXIT]; cx.font = `bold 12px ${FONT}`;
  cx.fillText(`STEP ${tut.step + 1} OF ${TUT_STEPS.length}`, x + 16, y + 24);
  cx.fillStyle = '#dff'; cx.font = `14px ${FONT}`;
  lines.forEach((l, i) => cx.fillText(l, x + 16, y + 46 + i * 20));
```

Lines 655 to 659. "STEP 3 OF 7" in gold, then the instruction text line by line.

```
  button('SKIP (ESC)', VW - 136, 14, 120, 36, startGame);
}
```

Lines 660 to 661. The skip button, which starts the real game straight away.

```
function drawTitle() {
```

Line 662. drawTitle() draws the start screen: rippling rings, the name, a one-line story and two buttons.

```
  cx.globalCompositeOperation = 'lighter';
  for (let i = 0; i < 4; i++) {
    const r = (t * 140 + i * 110) % 440;
    cx.globalAlpha = 0.35 * (1 - r / 440);
    cx.strokeStyle = COL[WALL]; cx.lineWidth = 2;
    cx.beginPath(); cx.arc(VW / 2, VH / 2 - 40, r, 0, Math.PI * 2); cx.stroke();
  }
  cx.globalCompositeOperation = 'source-over'; cx.globalAlpha = 1;
```

Lines 663 to 670. Four rings that spread out from the centre and start again. The % 440 (remainder after dividing by 440) is what makes them loop.

```
  cx.textAlign = 'center'; cx.fillStyle = '#bff';
  cx.font = `bold ${Math.min(96, VW / 5)}px ${FONT}`;
  cx.fillText('ECHO', VW / 2, VH / 2 - 20);
```

Lines 671 to 673. The game's name, sized down on narrow screens.

```
  cx.font = `15px ${FONT}`; cx.fillStyle = 'rgba(170,255,255,0.75)';
  wrap('You are blind. Sound is sight. Something else is listening.', VW - 32)
    .forEach((l, i) => cx.fillText(l, VW / 2, VH / 2 + 18 + i * 20));
```

Lines 674 to 676. The tagline, wrapped so it fits on phones.

```
677   const bw = 150, gap = 16, by = VH / 2 + 64;
678   button('PLAY', VW / 2 - bw - gap / 2, by, bw, 44, startGame, true);
679   button('TUTORIAL', VW / 2 + gap / 2, by, bw, 44, startTutorial);
```

Lines 677 to 679. PLAY and TUTORIAL buttons side by side, centred.

```
680   cx.textAlign = 'center'; cx.font = `12px ${FONT}`; cx.fillStyle = 'rgba(170,255,255,0.55)';
681   cx.fillText('SPACE to play  ·  T for tutorial  ·  best level ' + best, VW / 2, by + 72);
682   cx.fillText('headphones recommended', VW / 2, by + 92);
683 }
```

Lines 680 to 683. Keyboard hints, your best level, and a tip about headphones.

```
684 function drawGameOver() {
685   cx.textAlign = 'center';
686   cx.fillStyle = RED; cx.font = `bold ${Math.min(64, VW / 7)}px ${FONT}`;
687   cx.fillText('GAME OVER', VW / 2, VH / 2 - 40);
688   cx.fillStyle = 'rgba(200,255,255,0.85)'; cx.font = `15px ${FONT}`;
689   cx.fillText(`You reached level ${level}.  Best: ${best}`, VW / 2, VH / 2);
690   const bw = 150, gap = 16, by = VH / 2 + 30;
691   button('TRY AGAIN', VW / 2 - bw - gap / 2, by, bw, 44, startGame, true);
692   button('MENU', VW / 2 + gap / 2, by, bw, 44, () => { state = 'title'; });
693   cx.textAlign = 'center'; cx.font = `12px ${FONT}`; cx.fillStyle = 'rgba(170,255,255,0.55)';
694   cx.fillText('SPACE to try again  ·  ESC for menu', VW / 2, by + 72);
695 }
```

Lines 684 to 695. drawGameOver() draws the game over screen: how far you got, and buttons to try again or go back to the menu.

17. The game loop

```
697 let last = performance.now();
```

Line 697. last is the time of the previous frame, in milliseconds.

```
698 function frame(now) {
```

Line 698. frame() runs once per screen refresh (usually 60 times a second).

```
699   const dt = Math.max(0, Math.min(0.05, (now - last) / 1000));
```

Line 699. Work out how many seconds have passed since the last frame. It is capped at 0.05 so a slow frame or a hidden tab does not make everything jump, and it can never go below zero.

```
700   last = now; t += dt;
```

Line 700. Remember this frame's time and move the game clock forward.

```
701   update(dt);
702   draw();
703   requestAnimationFrame(frame);
704 }
```

Lines 701 to 704. Update the game, draw it, and ask the browser to call frame() again at the next refresh.

```
705 requestAnimationFrame(frame);
```

Line 705. Start the loop.

```
706 </script>
707 </body>
708 </html>
```

Lines 706 to 708. End of the script, the body and the page.

Where to go next

Now that you know how every line works, here are some friendly changes to try:

- **Easier or harder:** change the numbers in `HEAR` (how far creatures hear) or the hunting speed in `moveEnemies()`.
- **More lives:** change `MAX_LIVES`. The hearts update automatically.
- **A new colour scheme:** edit the `COL` line. The colours use HSL: hue (0 to 360 round the colour wheel), saturation and lightness.
- **Your own tutorial room:** redraw `TUT_MAP`. Keep every row the same length, surround it with #, and include one P, one C and one X.
- **A new sound:** add a line to `sfx` using `tone()`, then call it from wherever it should play.

Have fun with it.